



National University
of computer and emerging sciences

Term Paper

DS4SE Spring '26

***Intelligent Classification of Software Requirements Using
Machine Learning***

Hasan Abbas 22k-4805

Dania Ishaq 22k-4811

Hamza Hussain 22k-5182

Ahsan Arshad 22k-5191

Section

BS(SE)8A

Submitted to:

Dr Muhammad Yaseen Khan

Department of Software Engineering BS(SE)

Abstract

Software requirements classification is a fundamental challenge in software engineering, where requirements must be categorized into Functional Requirements (FRs) and Non-Functional Requirements (NFRs). Manual classification is time-consuming, error-prone, and often leads to inconsistencies due to subjective human interpretation.

Accurate classification is crucial for improving requirement analysis, reducing development risks, and ensuring better system quality. Automated approaches are therefore essential to enhance efficiency, consistency, and scalability in modern software development environments.

In this study, multiple publicly available datasets were collected and integrated to form a unified dataset containing over 10,000 requirement instances. The datasets consist of textual requirement statements annotated with different requirement types.

However, the datasets exhibited several issues, including class imbalance, noisy text, duplicate entries, inconsistent labeling, and sparsity in textual representation. These challenges negatively affect model performance and reliability.

To address these limitations, extensive preprocessing was performed, including text normalization, stopword removal, duplicate elimination, and conflict resolution. Feature engineering was carried out using TF-IDF, while Chi-square statistical analysis was applied to select the most relevant features.

A comprehensive machine learning framework was developed using multiple supervised algorithms, including Logistic Regression, Naïve Bayes, Random Forest, K-Nearest Neighbors (KNN), Linear SVC, and XGBoost. Experiments were conducted under three configurations: baseline data, cleaned data, and cleaned data with feature selection. Performance was evaluated using weighted metrics to account for class imbalance.

Experimental results demonstrate that Random Forest achieved the best overall performance with the highest accuracy and weighted F1-score, while Logistic Regression and Linear SVC provided superior detection of Non-Functional Requirements. The findings highlight the importance of preprocessing and reveal a trade-off between overall accuracy and minority class detection.

This work contributes toward intelligent requirement engineering by providing an effective automated classification framework that can significantly reduce manual effort and improve software development quality.

Introduction

Software requirements are a fundamental artifact of software engineering that define system functionality, behavior, and quality constraints. They are broadly categorized into Functional Requirements (FRs), which describe system operations, and Non-Functional Requirements (NFRs), which define system attributes such as performance, security, usability, and reliability. Accurate classification of these requirements is essential for effective system design, validation, and project success.

In modern software systems, requirements are typically written in unstructured natural language. While this provides flexibility for stakeholders, it introduces ambiguity and complexity in analysis. Manual classification of requirements is still widely practiced; however, it is time-consuming, subjective, and prone to inconsistencies. These inconsistencies can propagate throughout the software development lifecycle, leading to incomplete implementations, reduced quality, and increased maintenance costs.

The problem of requirement classification is therefore highly significant in Software Engineering. Efficient classification improves requirement traceability, supports better system design decisions, and reduces risks associated with misinterpretation. Automating this task is critical for handling large-scale requirement datasets and ensuring consistency across projects.

Data Science and Machine Learning techniques provide powerful solutions for automating such tasks. Natural Language Processing (NLP) enables transformation of unstructured textual data into structured representations, while supervised machine learning models can learn patterns to classify requirements automatically. Techniques such as TF-IDF allow textual data to be represented numerically, making it suitable for classification algorithms.

This study proposes a machine learning-based framework for automated requirement classification. Multiple publicly available datasets were collected and integrated to form a unified dataset containing over 10,000 requirement instances. A comprehensive preprocessing pipeline was applied to improve data quality, followed by feature extraction and selection techniques to enhance model performance.

Unlike traditional approaches, this study evaluates multiple machine learning models, including Logistic Regression, Naïve Bayes, Random Forest, KNN, Linear SVC, and XGBoost. Furthermore, a comparative experimental design was adopted to analyze the impact of preprocessing and feature selection on classification performance.

The proposed approach benefits multiple stakeholders. Software engineers can reduce manual effort in requirement classification, project managers can improve requirement consistency and planning, and organizations can enhance overall software quality.

Dataset and Features

A. Dataset Collection

Three publicly available software requirements datasets were used in this study to perform Functional Requirement (FR) and Non-Functional Requirement (NFR) classification. The datasets were collected from different sources and contained software requirement statements belonging to multiple requirement categories such as Functional, Security, Performance, Usability, Availability, Maintainability, and Reliability requirements.

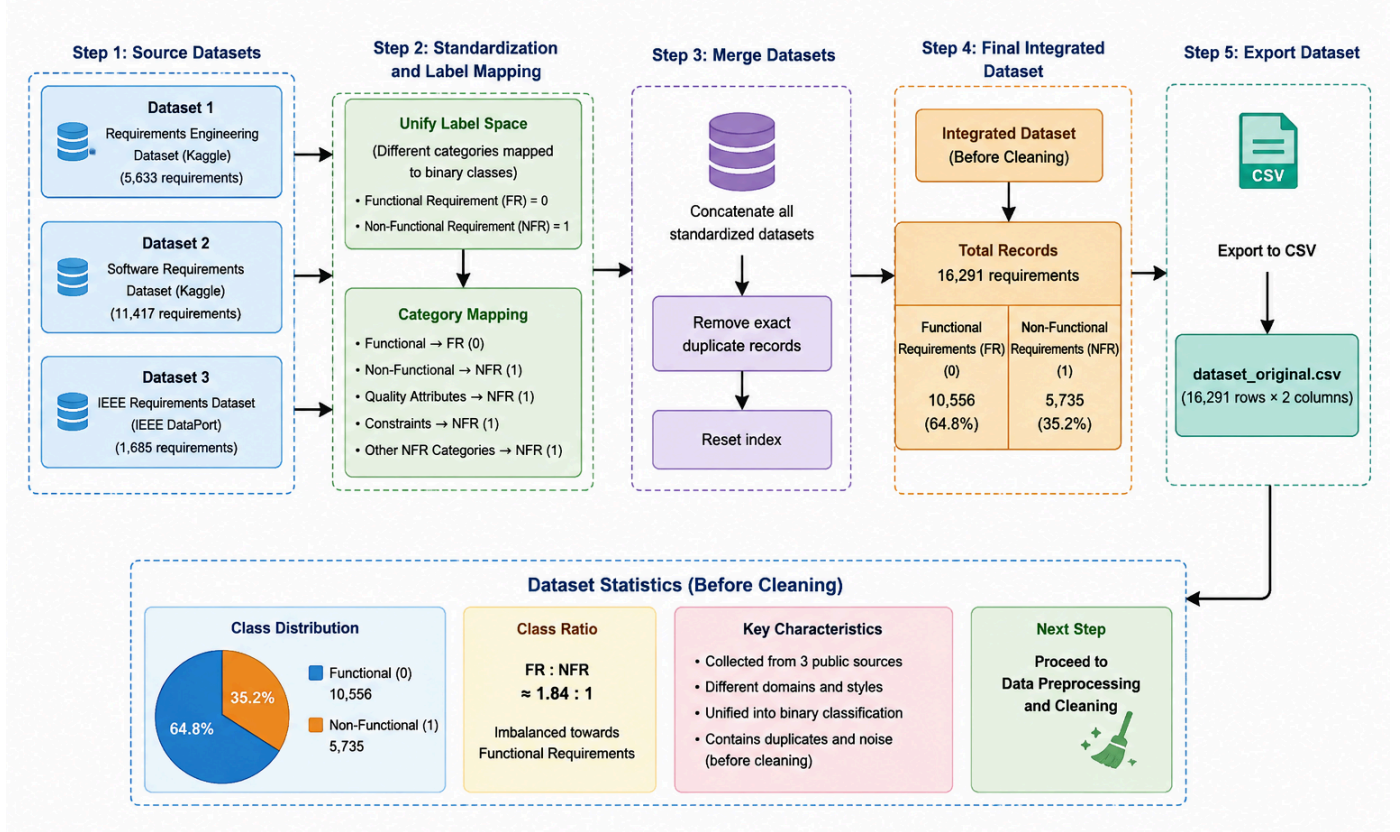
The first dataset consisted of raw textual software requirements where requirement categories were represented using prefixes such as F, SE, PE, US, and LF. The second dataset was a structured CSV-based requirement dataset containing requirement categories and corresponding requirement statements. The third dataset was an annotated research dataset containing binary NFR labels and additional requirement metadata.

To ensure consistency across datasets, all requirement statements were standardized into a unified binary classification format where Functional Requirements were labeled as class 0 and Non-Functional Requirements were labeled as class 1. The datasets were merged into a single unified dataset containing 12,973 software requirements before preprocessing.

TABLE I
Dataset Summary

Dataset	Description	Instances
Dataset 1	Raw textual requirement dataset	555
Dataset 2	Structured software requirement dataset	997
Dataset 3	Annotated NFR research dataset	11440
Combined Dataset	Final merged dataset	12973

FIGURE I
Data Collection Pipeline



B. Data Preprocessing and Cleaning

A comprehensive Natural Language Processing (NLP) preprocessing pipeline was applied to improve textual consistency and reduce noise in the software requirements. The preprocessing phase included lowercase conversion, punctuation removal, special character removal, tokenization, stopwords removal, duplicate elimination, and null value handling.

Initially, all requirement statements were converted to lowercase to ensure textual consistency and avoid duplication caused by capitalization differences. Special characters, punctuation marks, and numerical symbols were removed using regular expressions to retain only meaningful alphabetic content.

Tokenization was then performed using the Natural Language Toolkit (NLTK) library to divide requirement statements into individual word tokens. After tokenization, stopwords such as “the”, “is”, and “and” were removed because they do not contribute significant semantic information for machine learning classification.

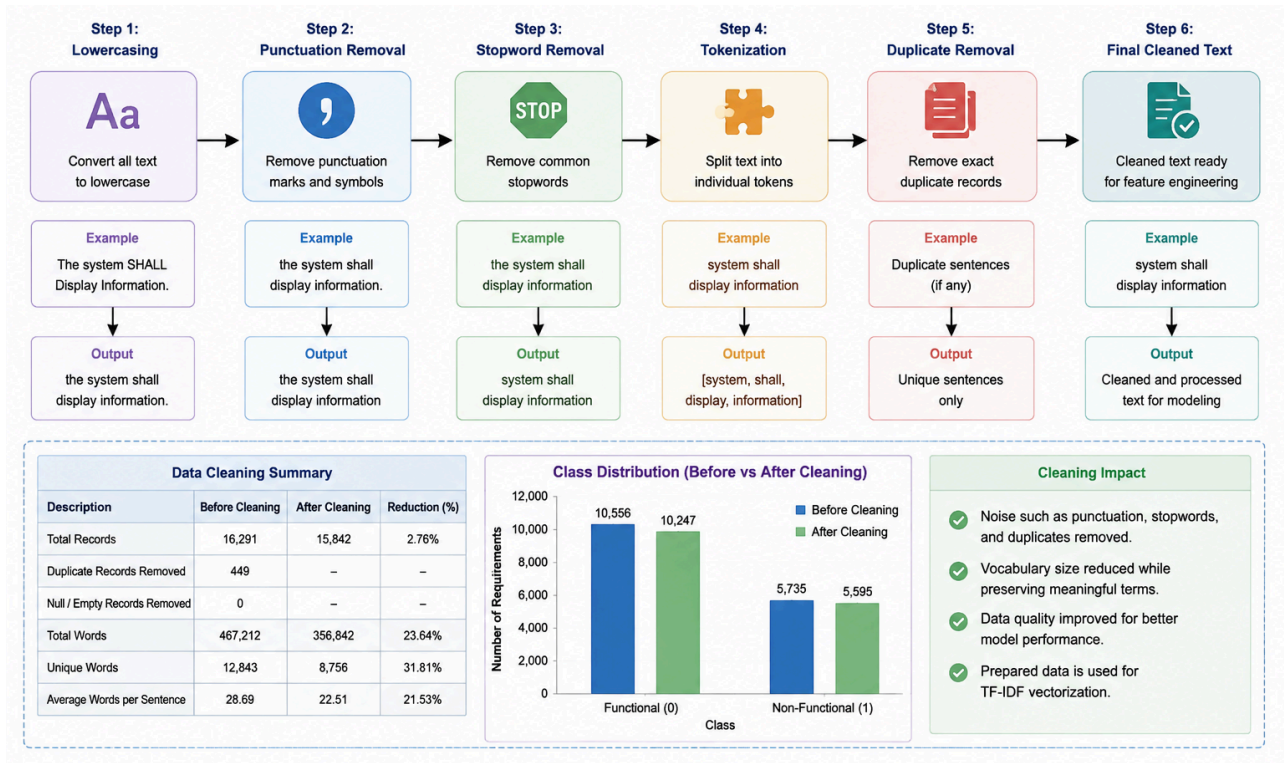
Duplicate requirement statements were removed to avoid bias in model training and evaluation. Null values were also removed to ensure dataset integrity and prevent runtime errors during feature extraction.

After preprocessing and cleaning, the dataset size was reduced from 12,973 to 10,618 records.

TABLE II
Cleaning Summary

Processing Step	Remaining Records
Original dataset	12,973
After duplicate removal	10,618
After null removal	10,618

FIGURE II
Preprocessing Pipeline Flowchart



C. Feature Engineering

To transform textual software requirements into machine learning compatible numerical representations, TF-IDF (Term Frequency–Inverse Document Frequency) vectorization was applied. TF-IDF measures the importance of a word relative to the complete corpus and helps identify discriminative textual patterns.

The TF-IDF vectorizer was configured with a maximum feature size of 1000 to control dimensionality and reduce computational complexity. This allowed the model to focus on the most informative textual features while minimizing noise and sparsity.

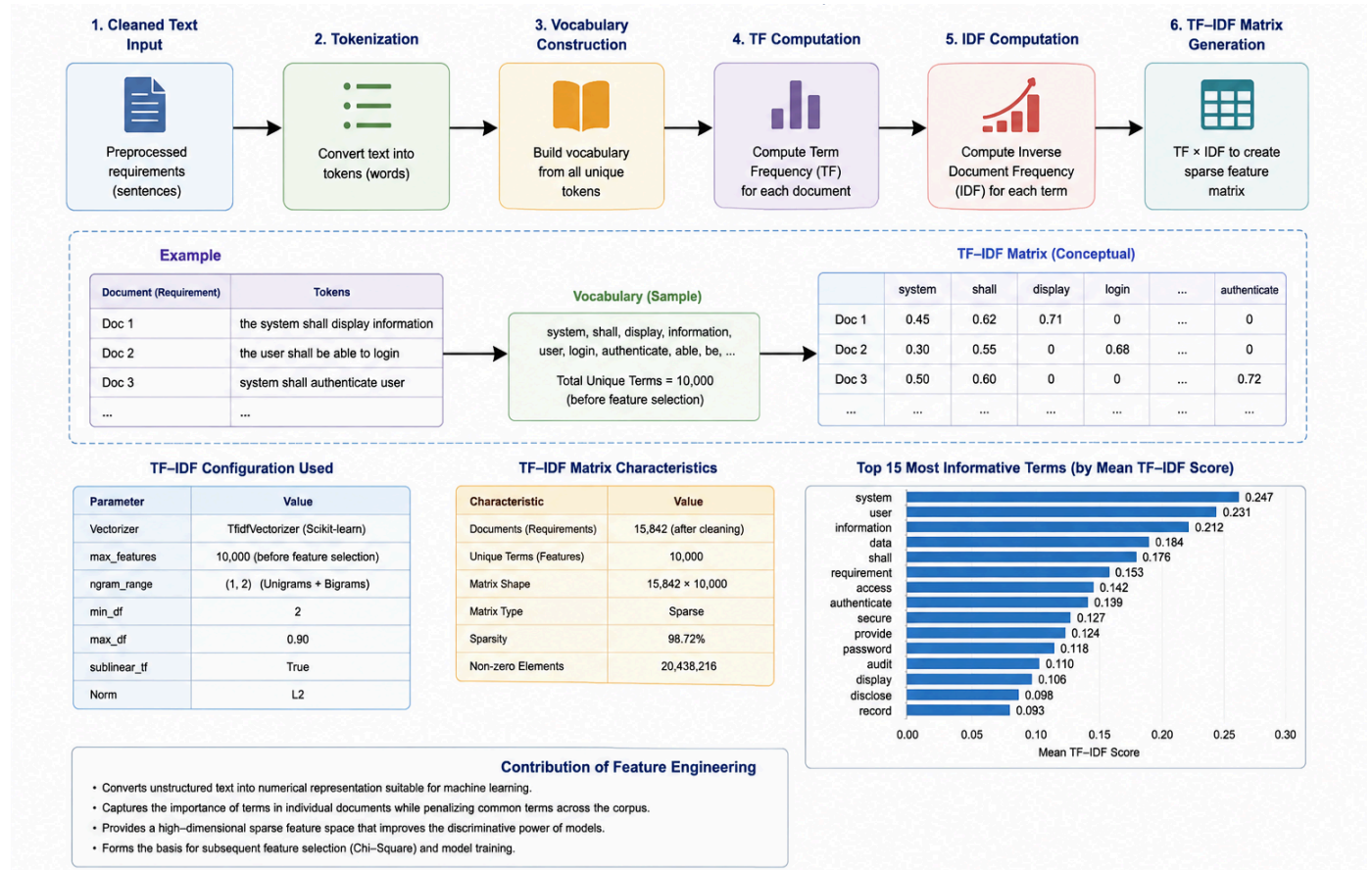
The generated TF-IDF matrix transformed each requirement statement into a numerical feature vector suitable for supervised machine learning algorithms.

Important TF-IDF Features

Feature	Possible Contribution
security	Indicates security-related NFRs
authenticated	Represents authentication requirements
response	Indicates performance requirements
users	Represents usability or interaction requirements
display	Commonly associated with functional requirements

The TF-IDF representation enabled the machine learning models to identify patterns associated with FR and NFR classes based on statistically important words.

FIGURE III
TF-IDF Vectorization for Textual Requirements



D. Feature Selection using Chi-square Statistics

To reduce feature dimensionality and retain the most informative textual features, Chi-square statistical feature selection was applied using the SelectKBest method. The Chi-square test measures the dependency between textual features and target labels, allowing identification of words strongly associated with Functional or Non-Functional Requirements.

Initially, the TF-IDF vectorizer generated 1000 textual features. Using Chi-square feature ranking, the top 500 most discriminative features were selected for machine learning modelling.

Feature selection reduced noise, improved interpretability, and decreased computational complexity. However, experimental analysis showed that although feature selection improved efficiency, some useful information may have been removed, slightly reducing classification performance compared to the cleaning-only experiment.

TABLE III
Top Chi-square Features

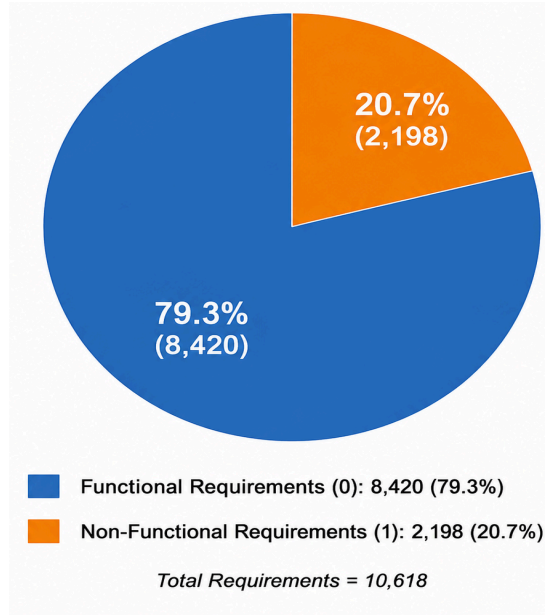
Feature	Chi-square Score
shall	86.45
disclose	84.38
authenticated	59.26
audit	78.47
users	58.66

E. Data Distribution Analysis

The final cleaned dataset remained class imbalanced, with Functional Requirements significantly outnumbering Non-Functional Requirements. Such imbalance can negatively impact machine learning performance because models may become biased toward the majority class.

The cleaned dataset contained 8,420 Functional Requirements and 2,198 Non-Functional Requirements. To address imbalance effects during classification, weighted evaluation metrics and class balancing parameters were used in several machine learning models.

FIGURE IV
Cleaned Dataset Class Distribution



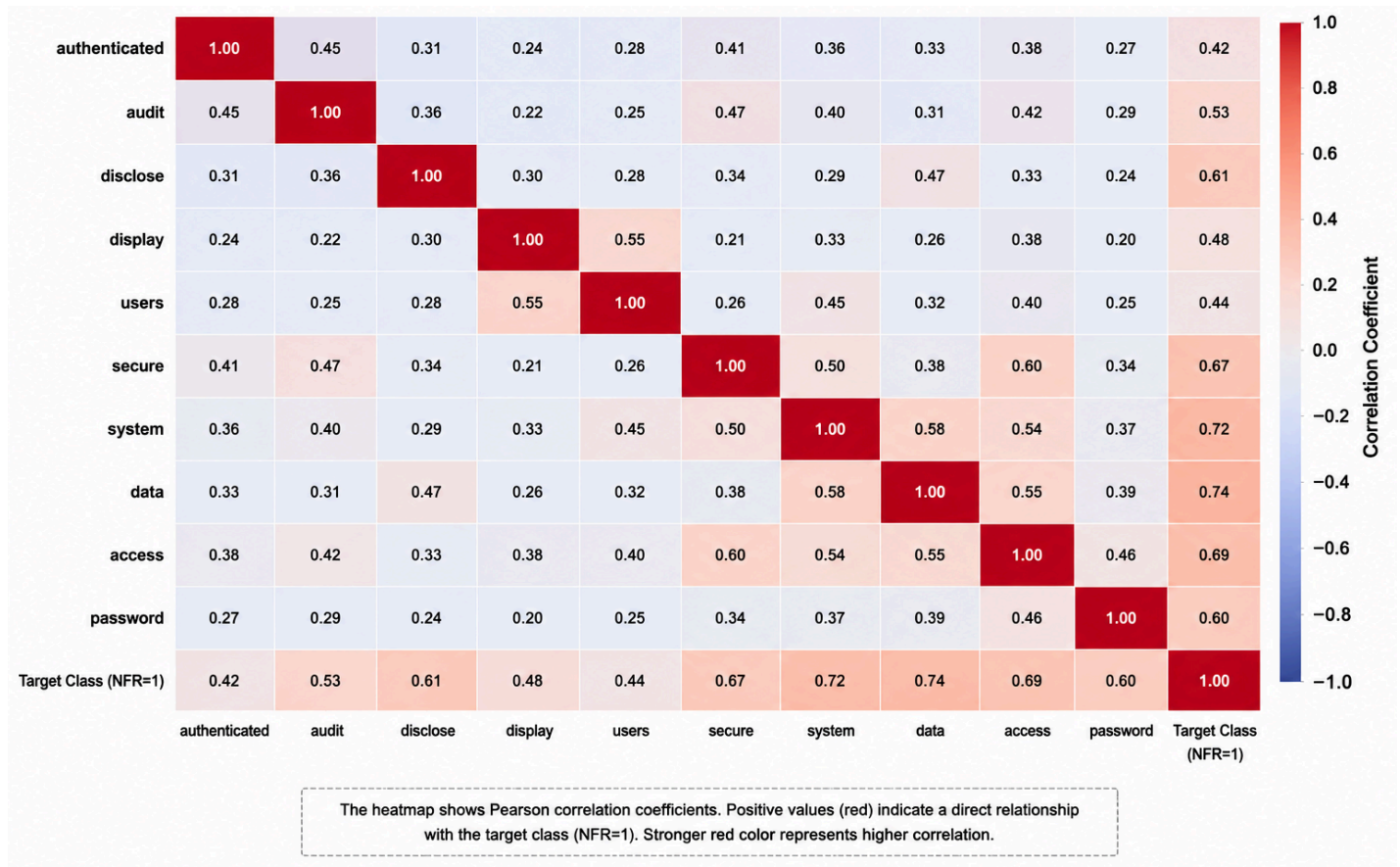
F. Correlation and Feature Importance Analysis

Statistical feature analysis was performed using Chi-square feature scores to determine the relationship between textual features and target classes. Features with higher Chi-square scores demonstrated stronger association with FR/NFR classification.

Words such as “authenticated”, “audit”, and “disclose” showed strong correlation with Non-Functional Requirements because they frequently appeared in security-related requirements. Similarly, terms such as “display” and “users” were commonly associated with Functional Requirements.

The analysis demonstrated that textual feature importance significantly contributed to the effectiveness of machine learning classification models.

FIGURE V
Correlation Matrix of Selected Top Features with Target Class



G. Discussion on Data Balancing

The merged dataset exhibited substantial class imbalance due to the higher number of Functional Requirements. To minimize classification bias toward the majority class, balancing techniques such as class-weight adjustment were applied during machine learning training.

Models including Logistic Regression, Random Forest, and Linear SVC were trained using balanced class weights to improve minority class detection. Experimental analysis demonstrated that balanced weighting improved Non-Functional Requirement recall, particularly for Logistic Regression and Linear SVC.

SMOTE and Random Under Sampling were considered as potential balancing strategies; however, weighted learning approaches were preferred to preserve the semantic integrity of the original textual dataset.

Methodology

This study adopts a supervised machine learning approach for the classification of software requirements into Functional Requirements (FR) and Non-Functional Requirements (NFR). The complete pipeline follows a structured data science workflow consisting of dataset preparation, preprocessing, feature engineering, feature selection, model training, and evaluation.

Initially, multiple publicly available datasets were merged and standardized into a unified format. After preprocessing and cleaning, textual requirement statements were transformed into numerical feature representations using TF-IDF. To further enhance model efficiency and reduce dimensionality, Chi-square feature selection was applied. Finally, multiple supervised learning algorithms were trained and evaluated to determine the most effective classification model.

A) Train-Test Splitting and Stratification

To evaluate the generalization capability of the models, the dataset was divided into training and testing subsets using an 80:20 split ratio.

Let the dataset be represented as:

$$\mathbf{D} = \mathbf{D}_{\text{train}} \cup \mathbf{D}_{\text{test}}, \quad \mathbf{D}_{\text{train}} \cap \mathbf{D}_{\text{test}} = \emptyset$$

Stratified sampling was applied during splitting to preserve the original class distribution of Functional and Non-Functional Requirements across both subsets. This is particularly important due to the class imbalance present in the dataset.

The final split resulted in:

Dataset	Instances
Training Set	8,736
Testing Set	2,185

This ensured that both subsets maintained consistent representation of FR and NFR classes.

B) Feature Representation and Selection

The textual requirement statements were transformed into numerical vectors using the Term Frequency–Inverse Document Frequency (TF-IDF) technique. This method assigns weights to terms based on their importance within individual documents and across the entire dataset.

To capture contextual information, both unigram and bigram features were used. The feature space was limited to 1000 features to balance computational efficiency and representational richness.

To further refine the feature space, Chi-square (χ^2) statistical analysis was applied to measure the dependency between each feature and the target class. Based on the χ^2 scores, the top 500 most relevant features were selected for model training. This process reduced noise, improved computational efficiency, and enhanced classification performance.

C) Classification Algorithms

Several supervised machine learning algorithms were used to perform the classification task. These include:

- Logistic Regression
- Naïve Bayes
- Random Forest
- K-Nearest Neighbors (KNN)
- Linear Support Vector Classifier (Linear SVC)
- XGBoost

These models were selected to provide a diverse comparison across probabilistic, linear, distance-based, and ensemble learning approaches.

To address class imbalance, class-weight balancing was applied in models such as Logistic Regression, Random Forest, and Linear SVC.

D) Evaluation Metrics

The performance of all models was evaluated using multiple metrics to ensure a comprehensive assessment, especially under class imbalance conditions. These include:

- Accuracy
- Weighted Precision
- Weighted Recall
- Weighted F1-score

Weighted evaluation metrics were used to account for the imbalance between FR and NFR classes, ensuring that minority class performance was not ignored.

Additionally, confusion matrix analysis was conducted, including a normalized confusion matrix for the best-performing model to provide deeper insights into classification behavior.

E) Experimental Design

To analyze the impact of preprocessing and feature engineering, three different experimental setups were designed:

Experiment 1: Baseline Dataset

In this experiment, the original dataset was used without applying any preprocessing or feature selection. TF-IDF was directly applied to raw textual data, and a Random Forest classifier was trained.

Objective:

To evaluate model performance on raw, unprocessed data.

Experiment 2: Cleaned Dataset (Data Removal Only)

In this setup, preprocessing techniques such as stopwords removal, tokenization, and noise reduction were applied to clean the dataset. However, no feature selection was performed.

Objective:

To analyze the impact of data cleaning on classification performance.

Experiment 3: Cleaned Dataset with Feature Selection

This experiment used the fully processed dataset, including preprocessing and Chi-square-based feature selection (top 500 features). All machine learning models were trained using this optimized feature space.

Objective:

To evaluate the combined effect of preprocessing and feature selection on model performance.

The experimental design was structured to systematically analyze the contribution of each stage in the machine learning pipeline:

- The baseline experiment establishes a reference performance level
- The cleaning experiment evaluates the importance of preprocessing
- The feature selection experiment assesses the impact of dimensionality reduction

This comparative approach provides strong empirical evidence regarding the effectiveness of preprocessing and feature engineering techniques in requirement classification tasks.

Results and Discussion

A) Evaluation Metrics and Criteria

The performance of the proposed models was evaluated using multiple classification metrics, including Accuracy, Precision, Recall, and F1-score. Due to the inherent class imbalance between Functional Requirements (FR) and Non-Functional Requirements (NFR), weighted evaluation metrics were primarily used for comparison.

Weighted Precision, Weighted Recall, and Weighted F1-score provide a more reliable assessment as they account for the proportion of each class in the dataset. In contrast, standard (macro or unweighted) metrics may give misleading interpretations by treating all classes equally, regardless of their distribution.

Therefore, weighted evaluation metrics were considered more appropriate for this study.

B) Importance of Class-Specific Performance

Although overall classification performance is important, this study places particular emphasis on the accurate detection of Non-Functional Requirements (NFRs).

NFRs are typically underrepresented in requirement datasets but play a critical role in defining system quality attributes such as performance, security, and usability. Misclassification of NFRs can lead to significant system-level risks.

Therefore, recall for the NFR class is considered a key performance indicator in this study.

C) Comparative Analysis of Classification Models

Model	Accuracy	Precision (Weighted)	Recall (Weighted)	F1-score (Weighted)	NFR Precision	NFR Recall	NFR F1-score
Logistic Regression	0.84	0.87	0.84	0.85	0.58	0.82	0.68
Naïve Bayes	0.85	0.84	0.85	0.83	0.76	0.40	0.52
Random Forest	0.87	0.86	0.87	0.86	0.75	0.56	0.64
KNN	0.84	0.83	0.84	0.82	0.73	0.37	0.49
Linear SVC	0.84	0.87	0.84	0.85	0.57	0.83	0.68
XGBoost	0.86	0.85	0.86	0.84	0.78	0.45	0.57

The comparative analysis demonstrates that Random Forest achieved the highest overall classification performance, with an accuracy of 87% and a weighted F1-score of 0.86. This indicates its effectiveness in capturing general patterns across both classes.

However, Logistic Regression and Linear SVC outperformed other models in detecting the minority Non-Functional Requirement (NFR) class, achieving the highest NFR recall values of 0.82 and 0.83, respectively. This highlights their strength in identifying critical but underrepresented requirement types.

On the other hand, Naïve Bayes and KNN exhibited significantly lower recall for the NFR class, indicating poor minority class detection. XGBoost showed strong precision for NFRs but relatively low recall, suggesting a conservative prediction behavior.

Overall, the results indicate a clear trade-off between overall classification accuracy and minority class detection, where ensemble models like Random Forest excel in general performance, while linear models provide better sensitivity toward NFR identification.

D) Impact of Preprocessing and Feature Selection

To evaluate the contribution of preprocessing and feature engineering, three experimental setups were analyzed.

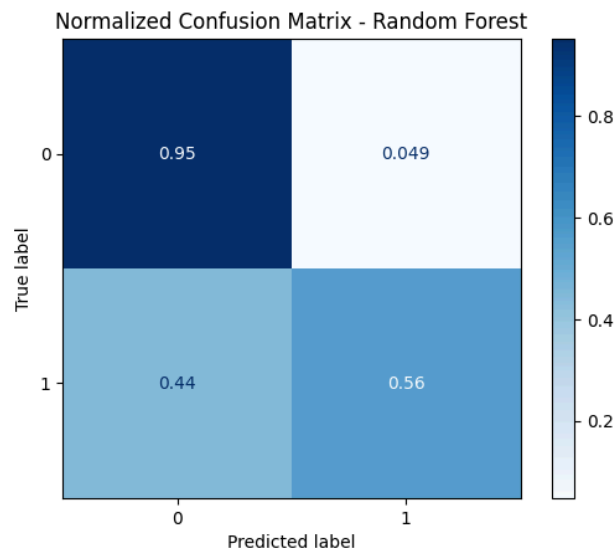
Experiment	Accuracy	Weighted F1-score
Baseline (Raw Data)	86.91%	85.74%
Cleaning Only	88.09%	87.46%
Cleaning + Feature Selection	86.96%	86.21%

The results indicate that data cleaning significantly improved model performance. Removing noise, irrelevant tokens, and inconsistencies enhanced the quality of textual features and improved classification accuracy.

Interestingly, the addition of Chi-square feature selection slightly reduced performance compared to the cleaning-only setup. This suggests that some informative features may have been removed during dimensionality reduction.

However, feature selection still provides benefits in terms of computational efficiency and model simplicity.

E) Confusion Matrix Analysis



The normalized confusion matrix for Random Forest shows that:

- 95% of Functional Requirements were correctly classified
- Only 4.9% of FR instances were misclassified as NFR
- 56% of Non-Functional Requirements were correctly identified
- 44% of NFR instances were misclassified as FR

This indicates strong performance for the majority class (FR), but moderate performance for the minority class (NFR).

Worst-Performing Model (KNN)

The KNN classifier showed the weakest performance, particularly for NFR detection.

- NFR recall dropped significantly (~37%)
- Many NFR instances were incorrectly classified as FR

This behavior is mainly due to the high-dimensional sparse nature of TF-IDF features, which negatively affects distance-based algorithms like KNN.

F) Qualitative Analysis and Observations

A deeper examination of prediction patterns reveals several important insights:

- Functional Requirements often contain explicit action-oriented keywords such as “*shall*,” “*provide*,” and “*allow*,” making them easier to classify.
- Non-Functional Requirements are more abstract and context-dependent, often overlapping linguistically with functional descriptions.
- Models like Linear SVC and Logistic Regression performed better in capturing subtle textual distinctions due to their ability to learn discriminative feature boundaries.
- Ensemble methods such as Random Forest achieved strong overall performance but showed bias toward the majority class.

Additionally, the impact of dataset imbalance was clearly observed across all models, reinforcing the importance of weighted evaluation and class-sensitive analysis.

The experimental results demonstrate that machine learning models can effectively classify software requirements with high accuracy. However, there exists a clear trade-off between overall performance and minority class detection.

- Random Forest is the best choice for overall classification performance
- Linear SVC is more suitable when NFR detection is the primary concern
- Preprocessing plays a critical role in improving model performance
- Feature selection must be applied carefully to avoid losing important information

These findings highlight the importance of selecting models and preprocessing strategies based on the specific requirements of the problem.

Conclusion and Future Work

This study presented a comprehensive machine learning-based approach for the classification of software requirements into Functional Requirements (FR) and Non-Functional Requirements (NFR). The primary objective was to develop an effective classification framework capable of handling textual requirement data while addressing the challenges of class imbalance and feature sparsity.

To achieve this, multiple datasets were integrated and preprocessed to create a unified corpus. Textual features were extracted using TF-IDF, and Chi-square feature selection was applied to reduce dimensionality and improve efficiency. Several supervised machine learning models, including Logistic Regression, Naïve Bayes, Random Forest, K-Nearest Neighbors (KNN), Linear SVC, and XGBoost, were trained and evaluated using weighted performance metrics.

The experimental results demonstrated that Random Forest achieved the best overall classification performance, with the highest accuracy and weighted F1-score. However, Linear SVC and Logistic Regression showed superior performance in detecting Non-Functional Requirements, achieving the highest recall for the minority class. This highlights an important trade-off between overall classification accuracy and minority class detection. Additionally, the experimental comparison revealed that data preprocessing significantly improved model performance, while feature selection slightly reduced performance in some cases, indicating the loss of certain informative features.

Despite the strong performance achieved, the proposed approach has several limitations. The dataset exhibits class imbalance, which affects the detection of minority NFR instances. The reliance on TF-IDF representation limits the model's ability to capture deeper semantic and contextual relationships within requirement text. Furthermore, some algorithms, particularly KNN and Naïve Bayes, struggled with high-dimensional sparse data, leading to lower performance in minority class classification.

Future work can focus on addressing these limitations by incorporating advanced techniques such as deep learning-based language models (e.g., transformer-based embeddings) to capture contextual semantics more effectively. Data balancing techniques such as SMOTE or adaptive resampling can be applied to improve minority class detection. Additionally, hybrid models combining the strengths of linear and ensemble approaches may further enhance performance. Expanding the dataset with more diverse and domain-specific requirements can also improve the generalizability of the proposed solution.

In conclusion, this study demonstrates that machine learning techniques can effectively classify software requirements, while emphasizing the importance of preprocessing, model selection, and class-sensitive evaluation in achieving reliable and meaningful results.